

Cursor 案例快速实践

本课程主要学习基于 Cursor 开发类 ChatGPT 智能问答平台的完整流程，先以 Plan 模式梳理并拆分需求为四个开发阶段，再切换至 Agent 模式分步开发：从初始化项目并测试接口健康度，到开发并优化前端 UI，再到对接 DeepSeek 模型测试接口数据格式、实现前后端打通，最后完成模型集成与功能优化，解决了推理过程展示、Markdown 格式渲染等问题，并可根据需求持续优化界面与体验。

1、开发目标

本章会带着大家基于 Cursor **从零开始**去开发一个**完整的 AI 项目**，做一个类似于 ChatGPT 的智能问答平台，ChatGPT 的界面如下：

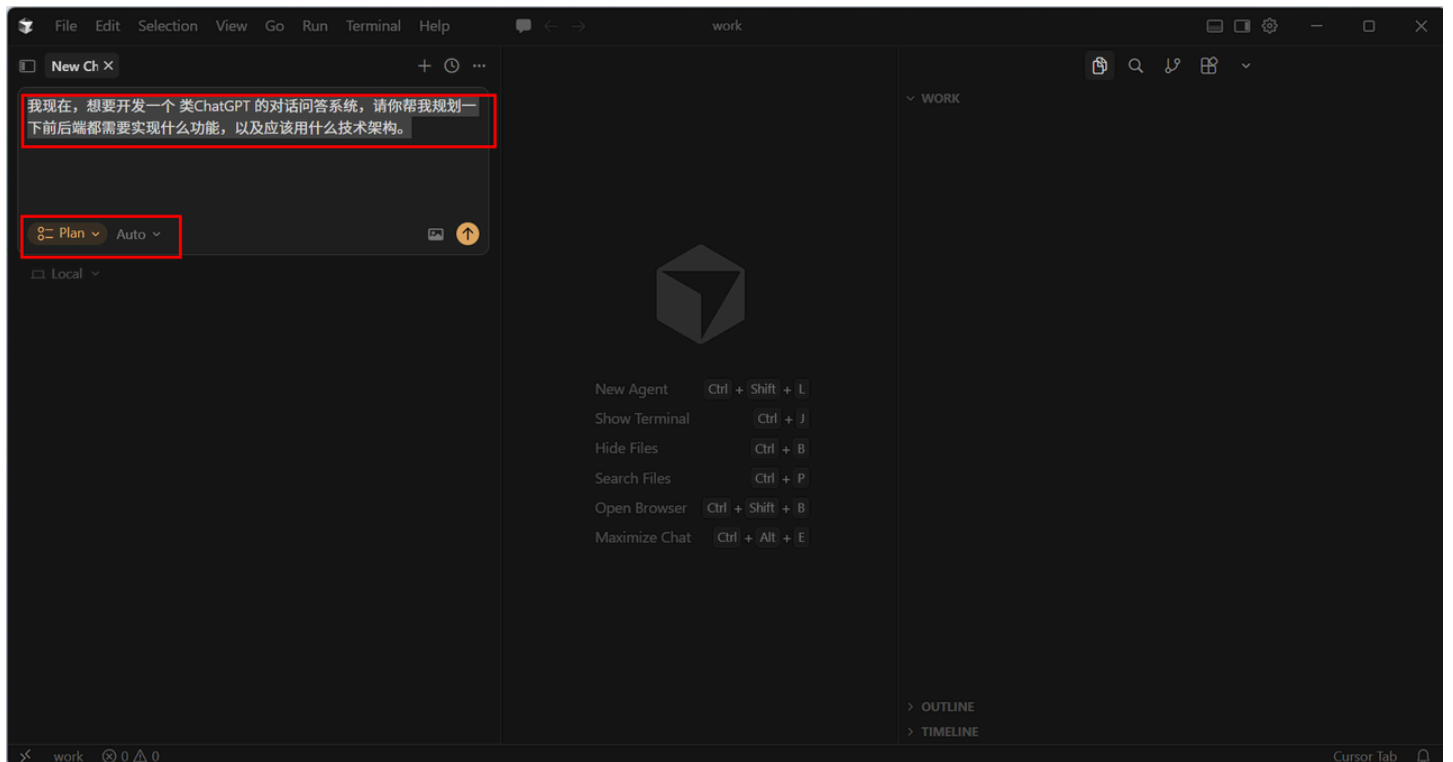


接下来整个项目开发，我们从**需求梳理 -> 拆分需求 -> 需求开发**，逐步落地。



2、梳理需求

我们开始项目的第一步，选择 Cursor 的工作模式：Plan 模式，通过 Plan 模式跟 Cursor 对齐需求。



1.参考提示词

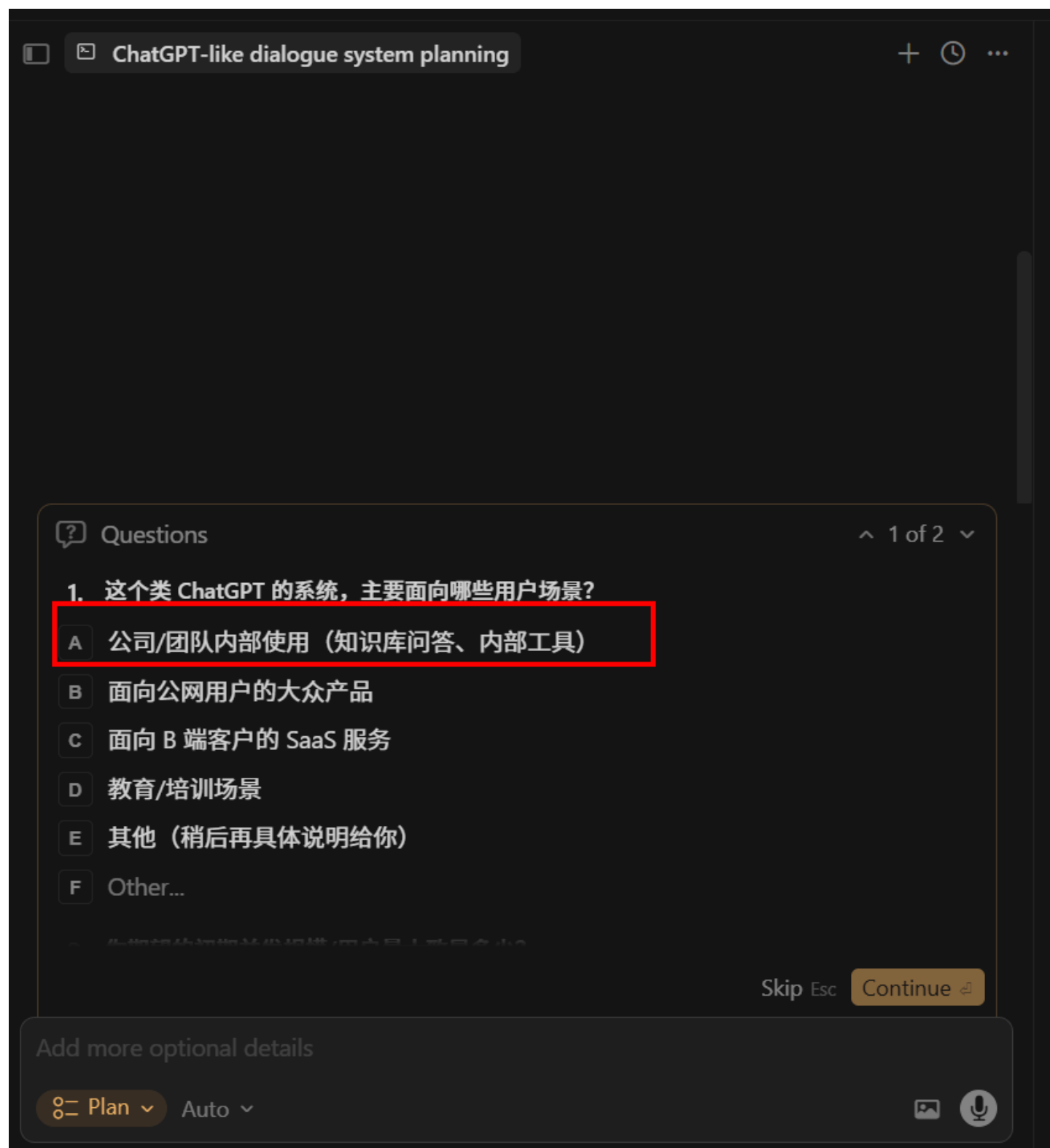
代码块

- 我现在想要开发一个类似 ChatGPT 的对话问答系统，请你帮我规划一下前后端都需要实现什么功能，以及应该用什么技术架构？

接下来 Cursor 会有一些的问题向你确认，你根据实际情况选择即可。

2.确定用户场景

因为不同的场景需要的技术架构也会不一样，我们当前就选择 A 场景即可。



ChatGPT-like dialogue system planning

Questions ^ 1 of 2 v

1. 这个类 ChatGPT 的系统，主要面向哪些用户场景？

- ☒ A 公司/团队内部使用（知识库问答、内部工具）
- ☐ B 面向公网用户的大众产品
- ☐ C 面向 B 端客户的 SaaS 服务
- ☐ D 教育/培训场景
- ☐ E 其他（稍后再具体说明给你）
- ☐ F Other...

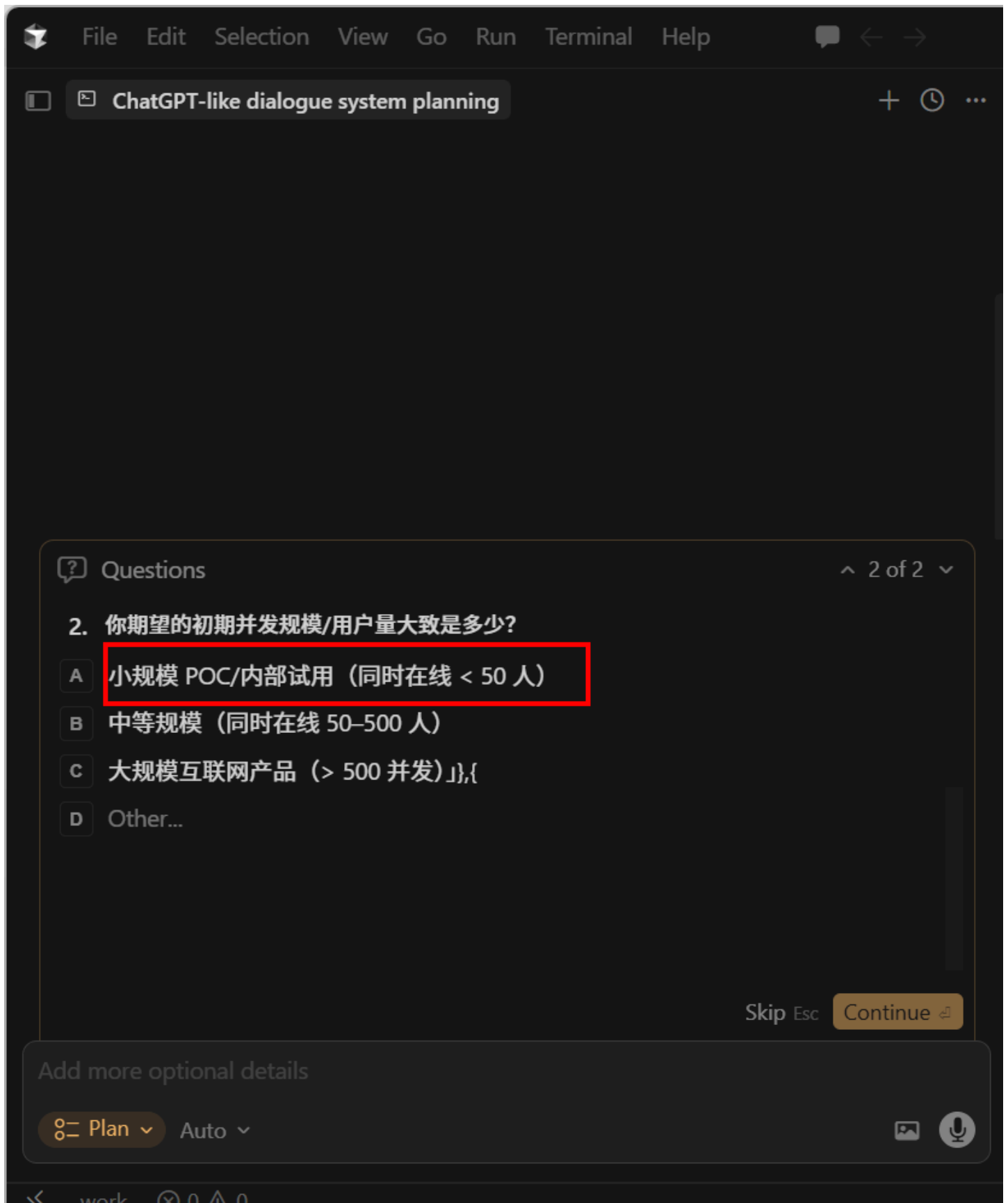
Skip Esc Continue

Add more optional details

Plan Auto

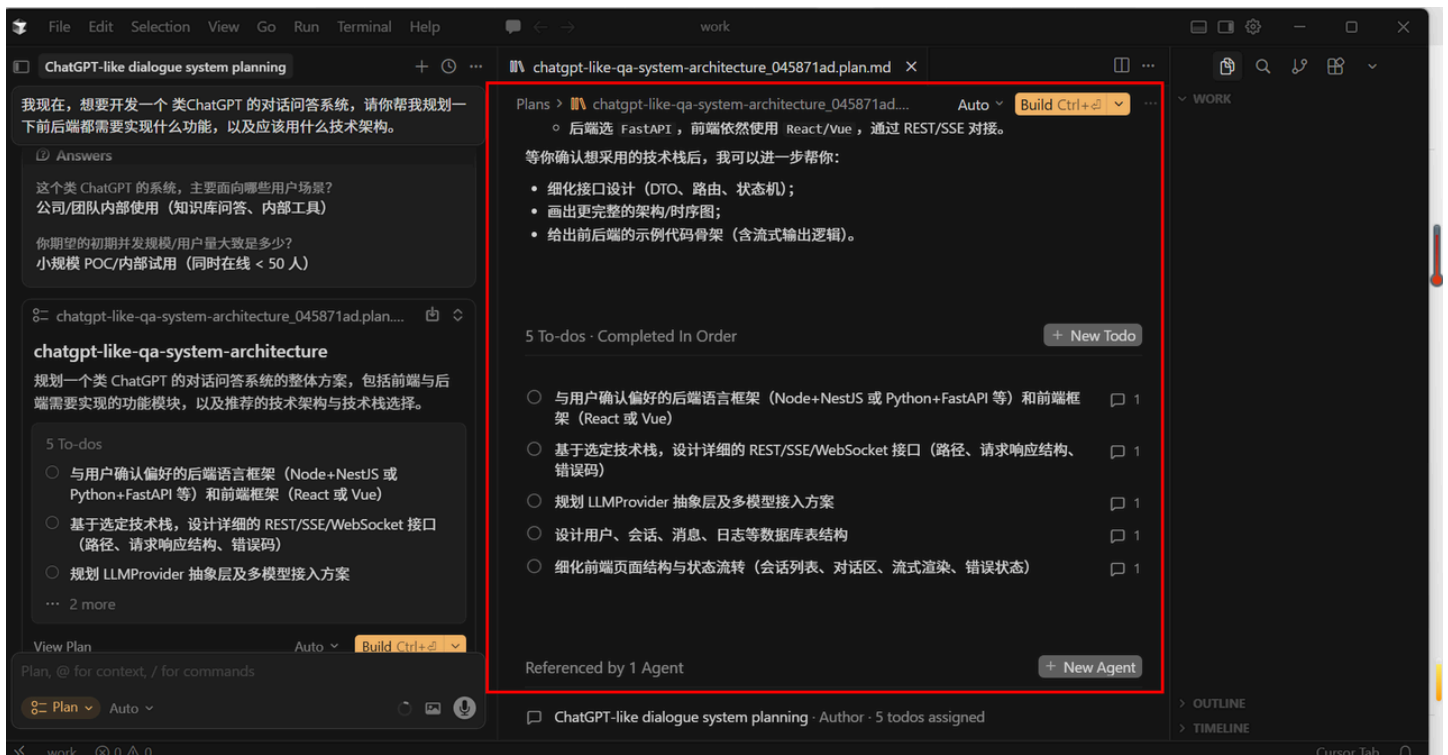
3.确认用户规模

不同的用户规模采用的架构也不一样，我们选择 A 即可。



4.继续跟进

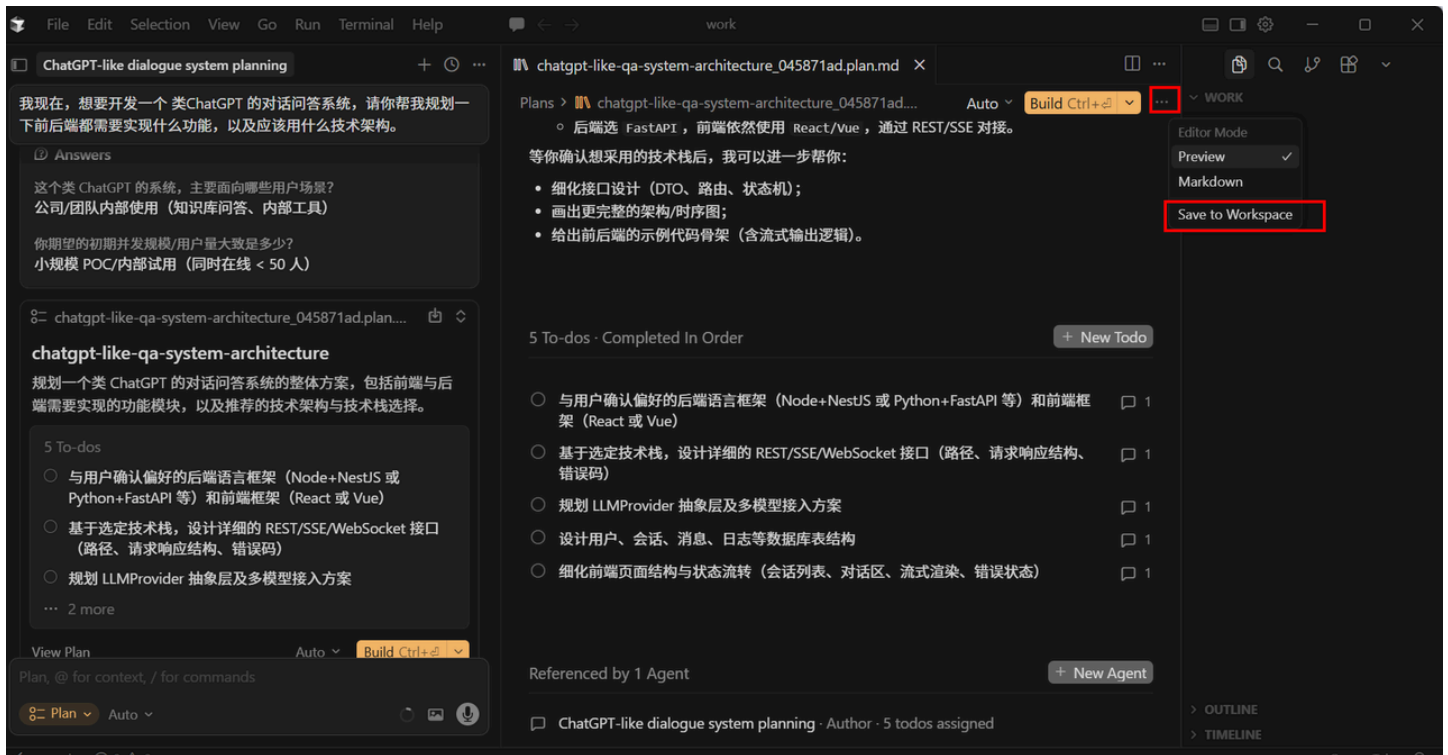
如果有其它确认问题，根据实际情况**继续确认即可**。运行完以后，会生成对应的技术文档。



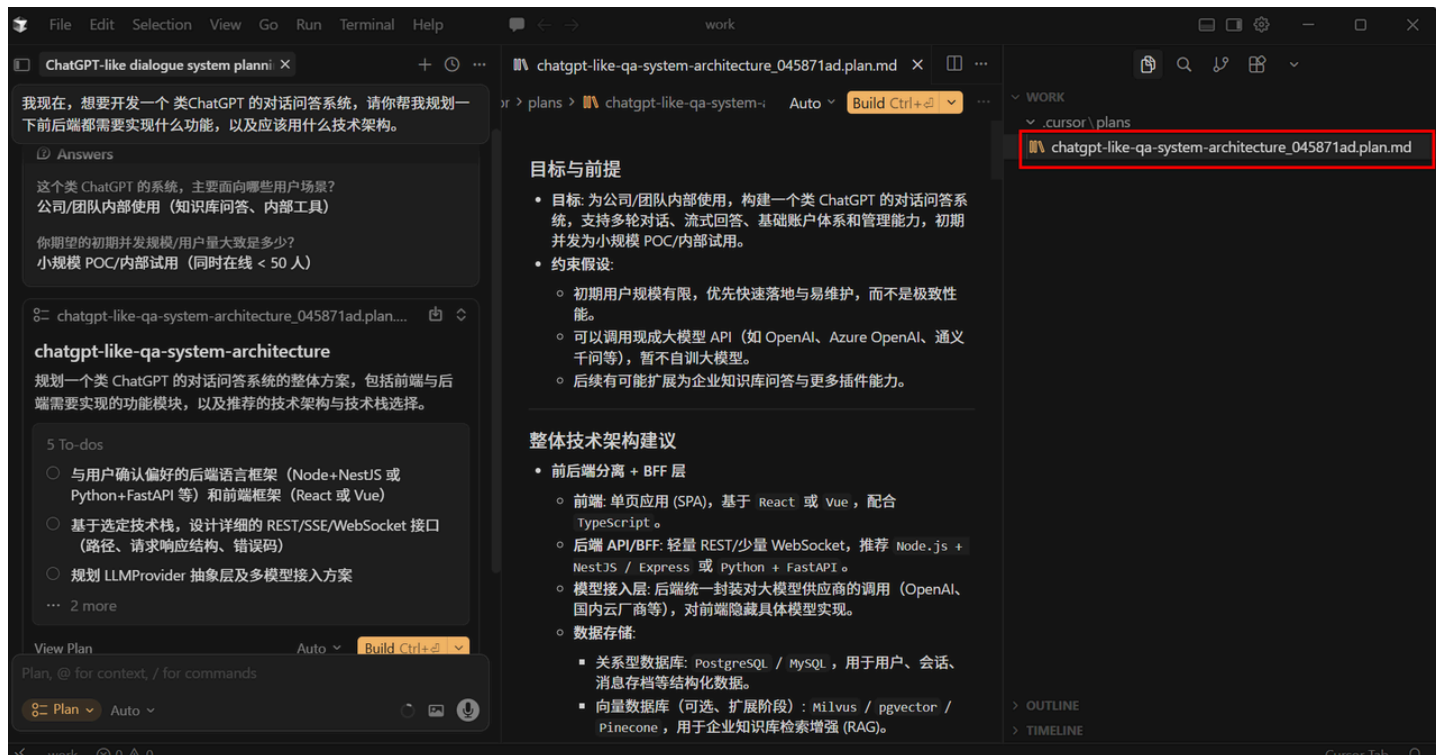
如果里面有需要改的，继续基于 Plan 模式跟 Cursor 对齐需求即可。

5.保存文档

对齐以后，保存一下需求文档。



我们就看到当前目录下有我们生成的技术文档了！



别着急执行后面的流程！

3、拆分需求

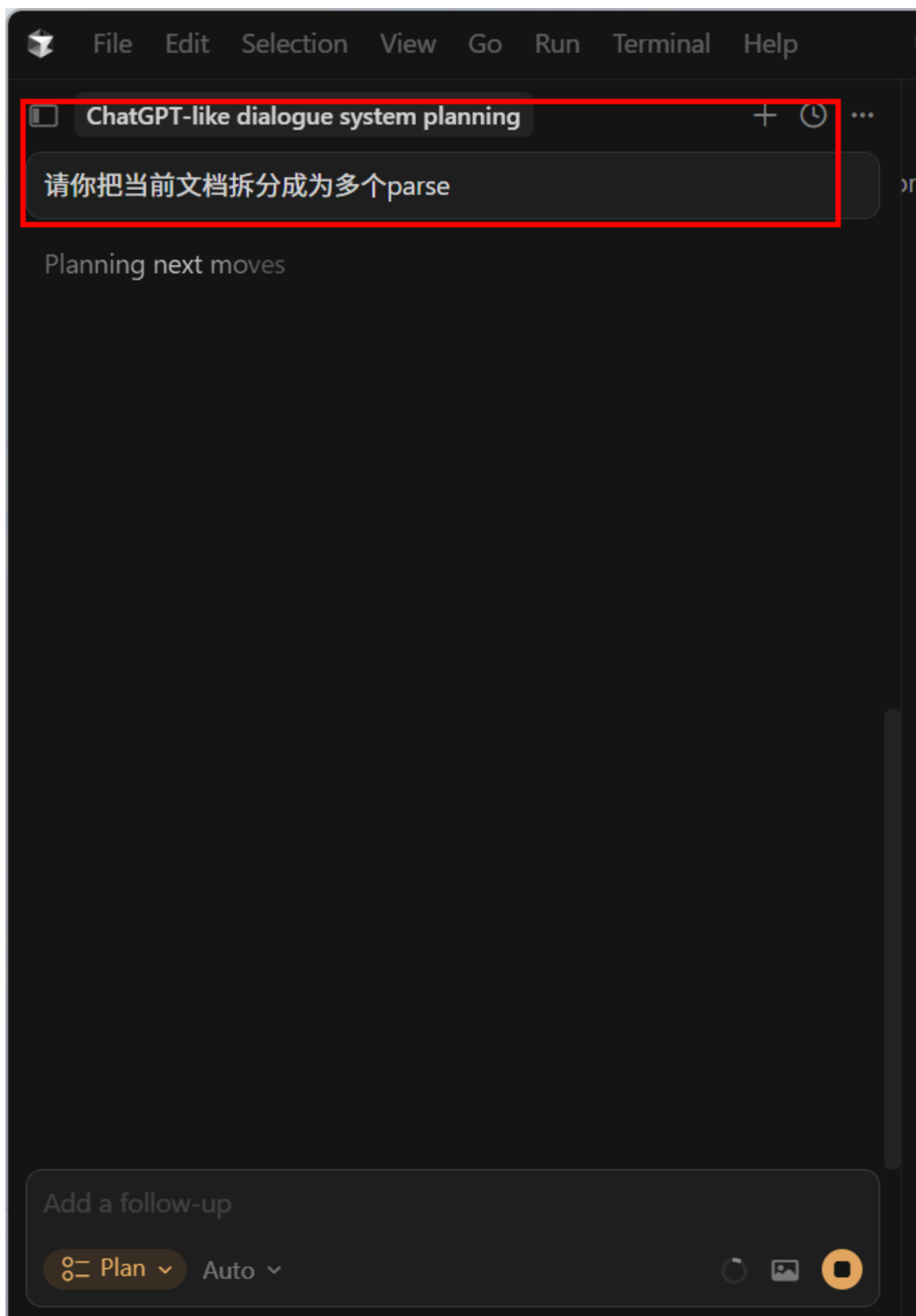
很多人把需求梳理完以后，就会选择让 Cursor 直接干，Cursor 直接干完以后，会非常容易出问题，所以这儿给大家分享我们团队的经验，把需求按步骤拆分，每一步都校验，这样最后基本不会有大问题！

1.拆分参考提示词

(注：parse这个我给写错了，应该是phase，阶段)

代码块

- 1 请你把当前文档拆分成多个 parse



2.生成结果

请你把当前文档拆分成为多个parse

Plan

Auto

Explored 1 file

下面把该文档按主题拆分成多个“parse”（每个 parse 独立可读、可复用）。我用 行号范围 标注对应原文位置，方便你后续复制/拆文件。

Parse 0：元信息与待办（Frontmatter）

- 范围：L1–L21
- 内容：计划名称、概述、todos

Parse 1：目标与前提

- 范围：L23–L30
- 内容：目标、约束假设

Parse 2：整体技术架构建议

- 范围：L33–L63
- 内容：前后端分离+BFF、分层、架构 mermaid 图

Parse 3：前端核心功能与技术栈

- 范围：L69–L103

Plan, @ for context, / for commands

Plan

Auto

可以调整一下 Cursor 给出来的开发计划。

3.参考提示词

代码块

- 1 帮我用如下的方式去规划我的开发任务：
- 2 parse 0：先初始化项目，创建完项目以后，测试前端后接口健康度

- 3 parse 1 : 先做前端UI开发
- 4 parse 2 : 做后端接口开发，接入DeepSeek模型，需要包含思考模型的过程，后面要在UI上展示。
- 5 parse 3 : 我需要集成DeepSeek推理模型和对话模型两种服务接口
- 6 按照这个调整一下。

FileEditSelectionViewGoRunTerminalHelp

ChatGPT-like dialogue system planning

parse 0：先初始化项目，创建完项目以后，测试前端后接口健康度

parse 1：先做前端UI开发

parse 2：做后端接口开发，接入 DeepSeek 模型，需要包含思考

chatgpt-like-qa-system-architecture_045871ad.plan.md +52 -44

```
todos:
- id: clarify-tech-stack
  content: 与用户确认偏好的后端语言框架 (Node+NestJS 或 Python)
- id: parse0-init-and-health
  content: 初始化前后端项目骨架；创建后端健康检查接口并在前端调用
  status: pending
```

已按你给的 parse 0–3 重新调整并写入当前计划文档，核心变化是：

- Parse 0：新增“初始化项目 + GET /api/health + 前端调用健康度测试”的落地步骤
- Parse 1：前端 UI 先行，并预留“思考/推理过程”展示区（可折叠）
- Parse 2：后端接入 DeepSeek，并将输出拆成 reasoning（思考过程）+ content（最终回答）两通道，支持流式推送给 UI 展示
- Parse 3：明确集成 DeepSeek 推理模型 + 对话模型 两种服务接口，并通过统一 Provider 抽象支持模型选择/切换

你现在直接在这份文档里看“迭代路线建议（里程碑）”那一节，就能看到按 parse 0–3 排序后的执行路径。

Plan, @ for context, / for commands

 Plan ▾ Auto ▾

确认没问题以后接下来按步骤去开发项目。

4、需求开发

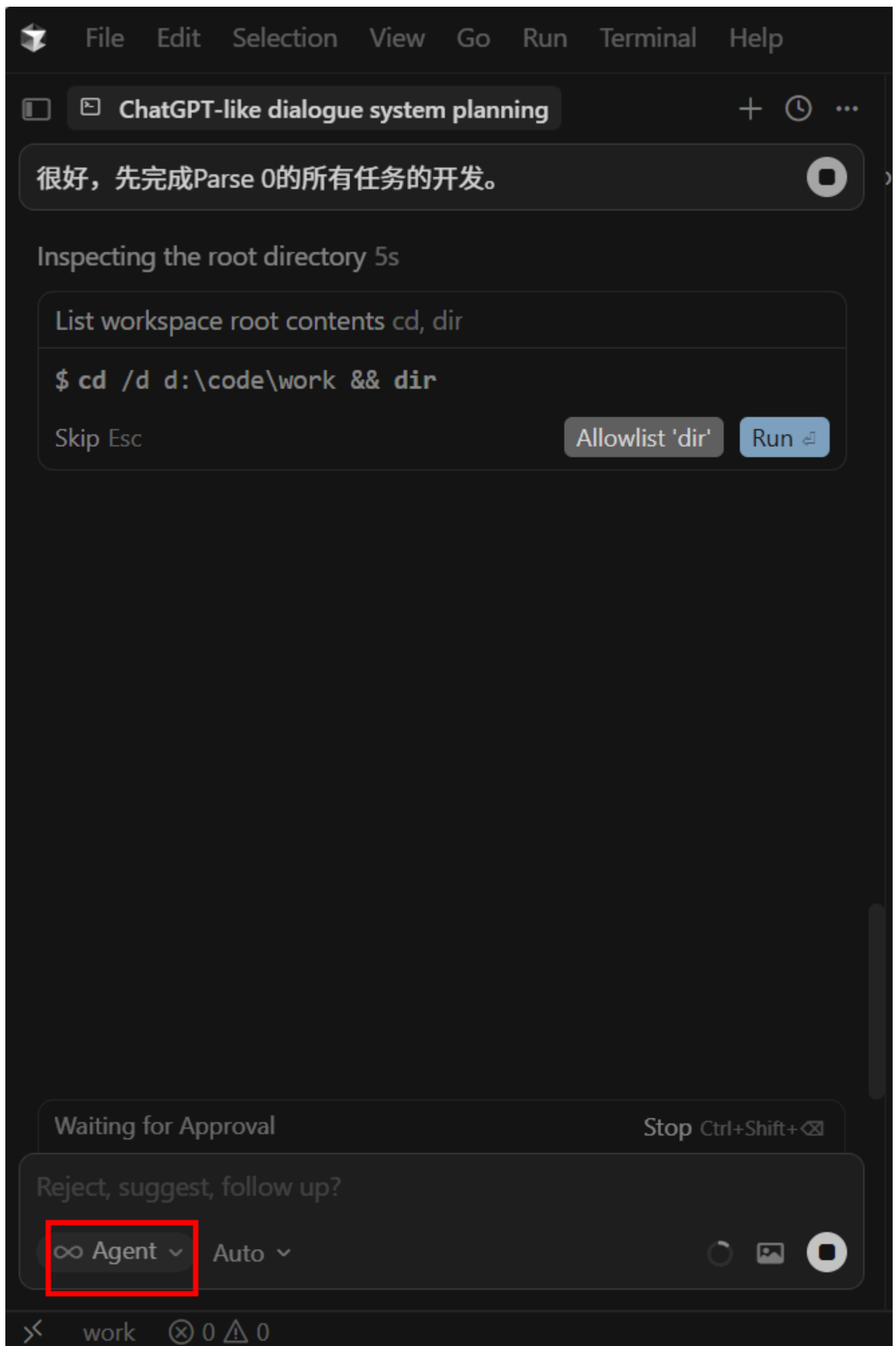
1.完成 parse 0 任务开发，构建好项目

把 Cursor 的工作模式切换 **Agent 模式**，然后完成 parse 0 的任务开发。

参考提示词：

代码块

```
1    很好，先完成Parse 0的所有任务的开发。
```



中间有需要交互的，有可能需要创建文件夹，需要安装依赖等，我们点 **run** 即可。

经过长时间的 Cursor 编码，我们看到结果，parse 0 阶段完成了！



接下来根据提示校验结果，进入 **cmd** 提示符命令窗口：

代码块

```
1 npm install
2 npm run dev
```

```
D:\code\work>npm run dev

> chatgpt-like-qa-system@1.0.0 dev
> concurrently -n backend,frontend -c magenta,cyan "npm --prefix backend run dev" "npm --prefix frontend run dev"

[backend] > backend@1.0.0 dev
[backend] > tsx watch src/index.ts
[frontend] > frontend@0.0.0 dev
[frontend] > vite
[frontend] 23:49:49 [vite] (client) Re-optimizing dependencies because vite config has changed
[frontend] Port 5173 is in use, trying another one...
[frontend] VITE v8.0.0 ready in 297 ms
[frontend] → Local: http://localhost:5174/
[frontend] → Network: use --host to expose
[backend] [dotenv@17.3.1] injecting env (0) from .env -- tip: 🛡️ prevent building .env in docker: https://dotenvx.com/p
ebuild
[backend] [backend] listening on http://localhost:8787
```

到浏览器输入：<http://localhost:5173/>

localhost

cursor官网

✂️ 🐱🏠 📄 🌐 ⚙️

8 📁 推荐 📁 购物 📁 娱乐 📁 办公

Parse 0: 前后端健康度

正常

前端通过 `/api/health` 验证后端接口可用性 (含代理/跨域链路)。

后端健康检查

重新检测

status	ok
service	backend
version	0.1.0
time	2026-03-16T15:50:56.407Z

看到页面显示：正常，第一步成功以后，接下来完成 parse 1；

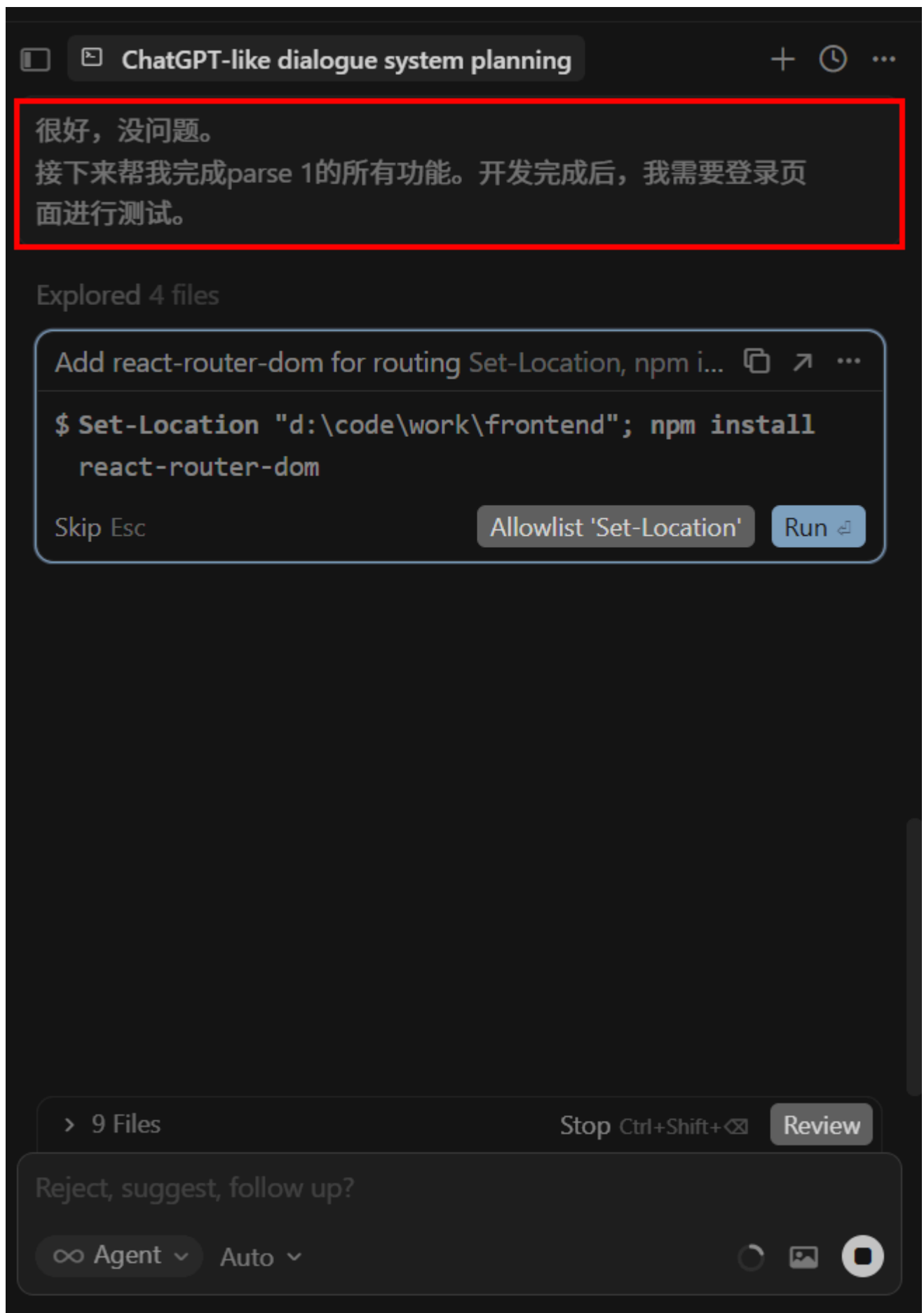
如果显示不正常，就继续交互，直到当前步骤正常，才继续进行下一步。

2.完成 parse 1 任务开发

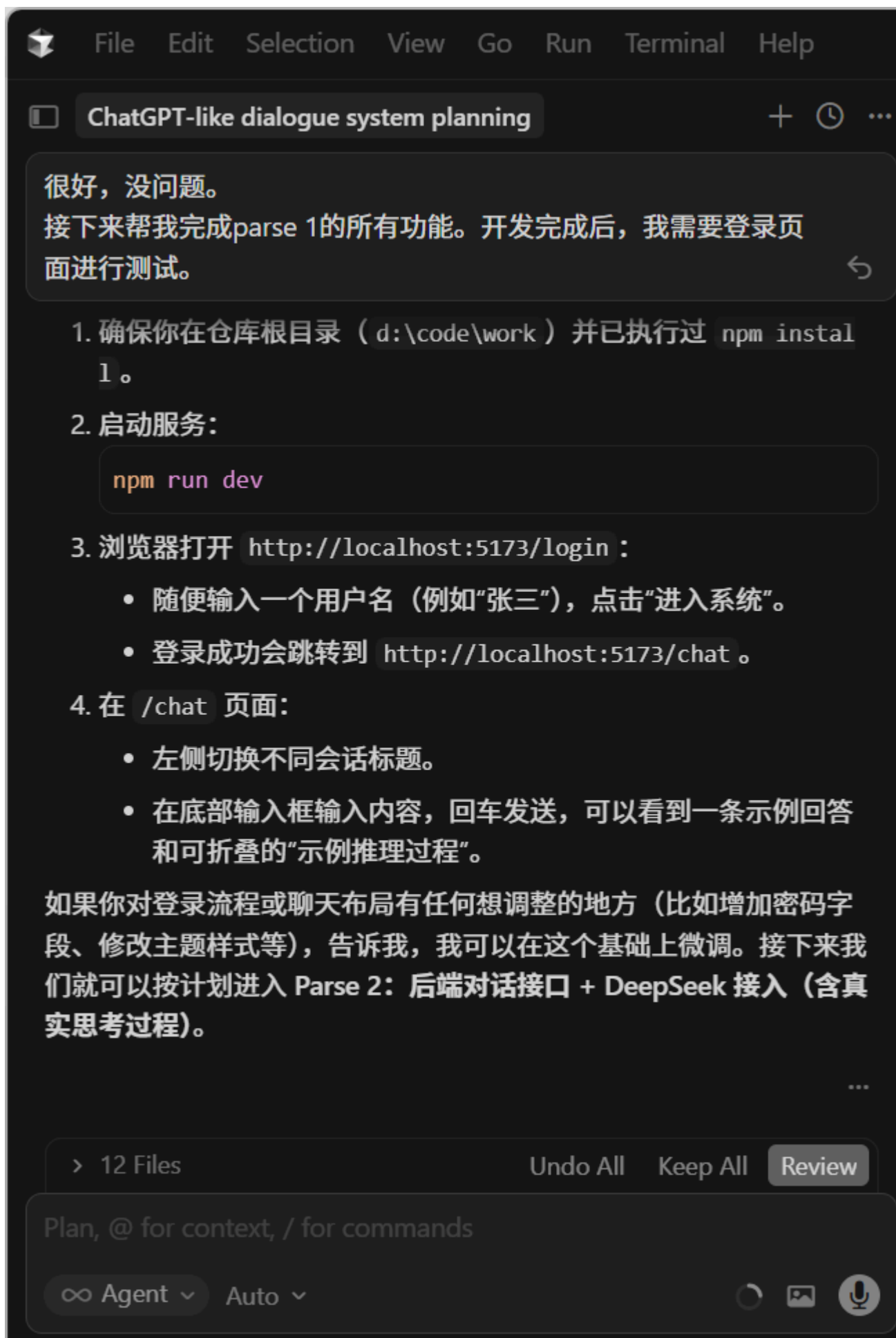
参考提示词：

代码块

1 接下来帮我完成parse 1的所有前端的功能。开发完成后，我需要登录页面进行测试。



很快已经完成开发，按照提示进行测试。



重启服务器，浏览器输入：<http://localhost:5173/>

CQ
内部对话系统

+ 新建会话

产品需求讨论 故障排查: 接口 500 DeepSeek 模型效果验证

产品需求讨论

登录用户: dd

DeepSeek 对话模型 (占位) 退出

AI

助手 (示例)

你好, 我是内部对话问答系统, 可以协助你进行需求澄清、代码解读和文档总结。

输入你的问题, 按
Enter 发送

☒ 自动展开思考过程 发送

我们尝试后可以登录, 功能没问题, 但是**页面太丑了**, 我们继续交互, 让 Cursor 进行优化。

参考提示词:

代码块

- 1 功能没问题, 但是现在页面长图中这样, 页面太丑了, 模仿ChatGPT的风格重新生成一个页面。



很好，没问题。

接下来帮我完成parse 1的所有功能。开发完成后，我需要登录页面进行测试。



1. 确保你在仓库根目录（`d:\code\work`）并已执行过 `npm install`。

2. 启动服务：

```
npm run dev
```

3. 浏览器打开 `http://localhost:5173/login`：

- 随便输入一个用户名（例如“张三”），点击“进入系统”。
- 登录成功会跳转到 `http://localhost:5173/chat`。

4. 在 `/chat` 页面：

- 左侧切换不同会话标题。
- 在底部输入框输入内容，回车发送，可以看到一条示例回答和可折叠的“示例推理过程”。

如果你对登录流程或聊天布局有任何想调整的地方（比如增加密码字段、修改主题样式等），告诉我，我可以在这个基础上微调。接下来我们就可以按计划进入 Parse 2：后端对话接口 + DeepSeek 接入（含真

> 12 Files

Undo All

Keep All

Review



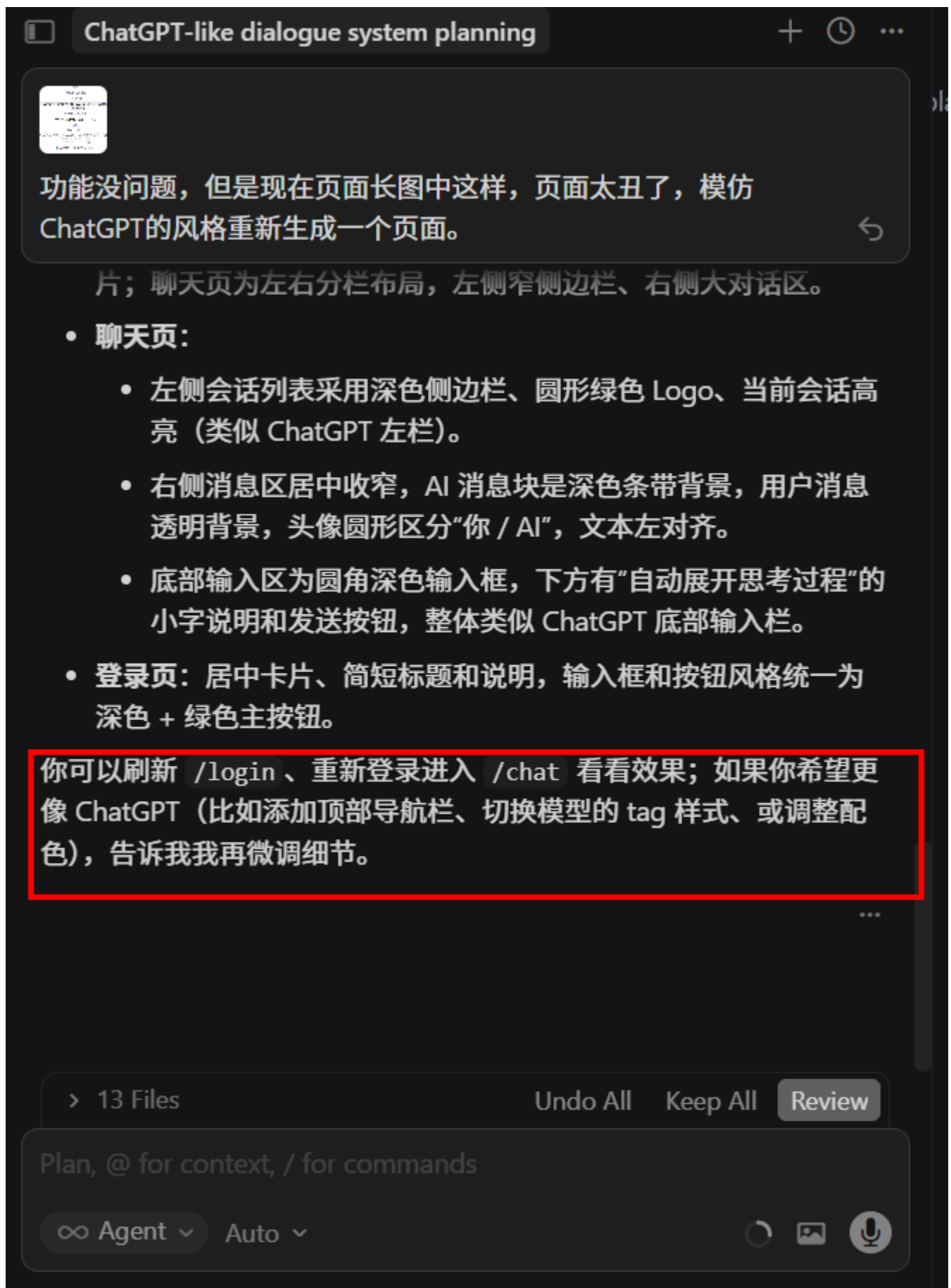
功能没问题，但是现在页面长图中这样，页面太丑了，模仿ChatGPT的风格重新生成一个页面。

∞ Agent ▾

Auto ▾



经过重新优化以后，我们重启服务，感受一下。



虽然还是丑，但是明显比前面好多了，大家可以继续交互，直到满意为止！



3.完成 parse 2 任务开发

在本阶段，我们需要完成后端核心代码的开发，并确保后端接口与前端实现数据互通。但如果直接让 Cursor 生成完整的后端业务代码，极易出现接口逻辑不完整、数据格式不统一、前后端联调适配性差等问题，大幅增加后续排查与整改的成本。

为此，这里分享一个关键的开发技巧：**暂缓直接生成业务代码**，优先让 Cursor 生成用于测试后端接口的标准化代码（如接口调用示例、参数校验逻辑、返回值验证脚本等），通过该测试代码先明确并固化后端接口的返回数据格式（包括字段定义、数据类型、嵌套结构、异常返回规范等），让 Cursor 精准感知后端数据输出的标准范式。

这一步的核心价值在于，提前锚定后端数据交互的基准，后续生成的后端业务代码与前端对接代码都会基于统一的格式规范，从源头规避前后端数据格式不兼容、接口调用失败等核心问题。

参考提示词：

代码块

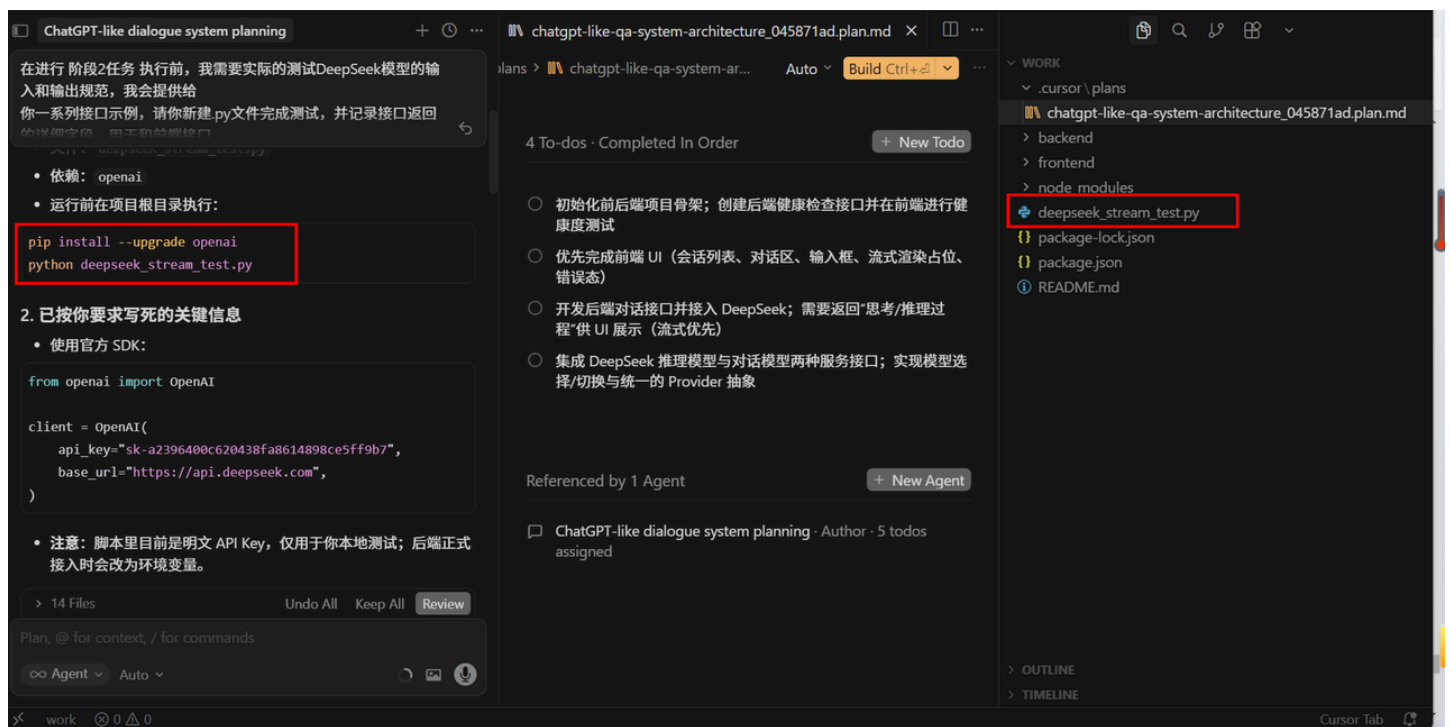
```
1  在进行 parse 2任务 执行前，我需要实际的测试DeepSeek模型的输入和输出规范，我会提供给
2  你一系列接口示例，请你新建.py文件完成测试，并记录接口返回的详细字段，用于和前端接口对接。
3  1. deepSeek 的 APi key 是 sk-408b4785bc5f482db70a8e996d4ee074，你可以在测试阶段显
   式
4  写在py文件中
5  2、DeepSeek官方的流式输出API，示例如下：
6  from openai import OpenAI
7  client = OpenAI(api_key="<DeepSeek API Key>",
   base_url="https://api.deepseek.com")
8
9  # Turn 1
10 messages = [{"role": "user", "content": "9.11 and 9.8, which is greater?"}]
11 response = client.chat.completions.create(
12     model="deepseek-reasoner",
```

```

13     messages=messages,
14     stream=True
15 )
16
17 reasoning_content = ""
18 content = ""
19
20 for chunk in response:
21     if chunk.choices[0].delta.reasoning_content:
22         reasoning_content += chunk.choices[0].delta.reasoning_content
23     else:
24         content += chunk.choices[0].delta.content
25
26 # Turn 2
27 messages.append({"role": "assistant", "content": content})
28 messages.append({'role': 'user', 'content': "How many Rs are there in the word
29 'strawberry'?"})
30 response = client.chat.completions.create(
31     model="deepseek-reasoner",
32     messages=messages,
33     stream=True
34 )
35 # ...
36
37 3. 首先，DeepSeek-v3.2是混合推理模型。请统一使用deepseek-chat接入，同时开启思考模
38 式的参数及规范如下：
39 如果您使用的是 OpenAI SDK，在设置 thinking 参数时，需要将 thinking 参数传入
40 extra_body 中：
41 response = client.chat.completions.create(
42     model="deepseek-chat",
43     # ...
44     extra_body={"thinking": {"type": "enabled"}}
45 )
46 请开始测试。

```

执行完成以后，按照需求，生成了一个测试文件，我们根据提示运行测试文件。



进入 cmd，执行测试脚本，复制脚本运行的结果给 Cursor，使它知道数据格式。

参考提示词：

代码块

- 1 按照目前的这些字段规范，让后帮我完成parse 2的所有的后端接口的开发。

代码块

- 1 按照你说的命令，执行了以后返回了如下内容，你可以参考如下内容，设计接口，使得前后端能对接上，接下来完成parse 2 的前端接口实际接入。
- 2 [chunk #22] has_reasoning=True has_content=False finish_reason=None
- 3 [chunk #23] has_reasoning=True has_content=False finish_reason=None
- 4 [chunk #24] has_reasoning=True has_content=False finish_reason=None
- 5 [chunk #25] has_reasoning=True has_content=False finish_reason=None
- 6 [chunk #26] has_reasoning=True has_content=False finish_reason=None
- 7 [chunk #27] has_reasoning=True has_content=False finish_reason=None
- 8 [chunk #28] has_reasoning=True has_content=False finish_reason=None
- 9 [chunk #29] has_reasoning=True has_content=False finish_reason=None
- 10 [chunk #30] has_reasoning=True has_content=False finish_reason=None
- 11 [chunk #31] has_reasoning=True has_content=False finish_reason=None
- 12 [chunk #32] has_reasoning=True has_content=False finish_reason=None
- 13 [chunk #33] has_reasoning=True has_content=False finish_reason=None
- 14 [chunk #34] has_reasoning=True has_content=False finish_reason=None
- 15 [chunk #35] has_reasoning=True has_content=False finish_reason=None
- 16 [chunk #36] has_reasoning=True has_content=False finish_reason=None
- 17 [chunk #37] has_reasoning=True has_content=False finish_reason=None
- 18 [chunk #38] has_reasoning=True has_content=False finish_reason=None
- 19 [chunk #39] has_reasoning=True has_content=False finish_reason=None

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

349 首先，问题：单词 "strawberry" 里有多少个字母 R？我需要统计字母 R 出现的次数，包括大小写
吗？单词通常是小写，所以我看小写形式 "strawberry"。仔细看单词：s-t-r-a-w-b-e-r-r-y。逐
个字母：s, t, r, a, w, b, e, r, r, y。所以字母 r 出现在第3位、第8位和第9位？实际上，我
们数一下：位置1: s, 2: t, 3: r, 4: a, 5: w, 6: b, 7: e, 8: r, 9: r, 10: y。所以有
三个 r。但确认一下，是字母 R 还是字母 R 不论大小写？因为题目是“字母 R”，可能指大写字母
R，但单词是小写的，所以应理解为字母 R 或 r 都算。通常这种问题统计的是该字母的出现次数。所
以答案是 3。

350

351 需要只给出数字并简单解释。所以我可以回答：3，因为单词 strawberry 中有三个字母 r。

352

353 但题目要求：“请只给出数字并简单解释。” 所以我应该先给出数字，然后解释。

354

355 在回答中，我可以说：3。然后解释：单词 strawberry 拼写为 s-t-r-a-w-b-e-r-r-y，其中字母
r 出现三次。

356

357 但确保不产生歧义。有时候人们可能错误地认为有两个 r，但实际有三个：一个在开头部分，两个在结
尾部分。是的。

358

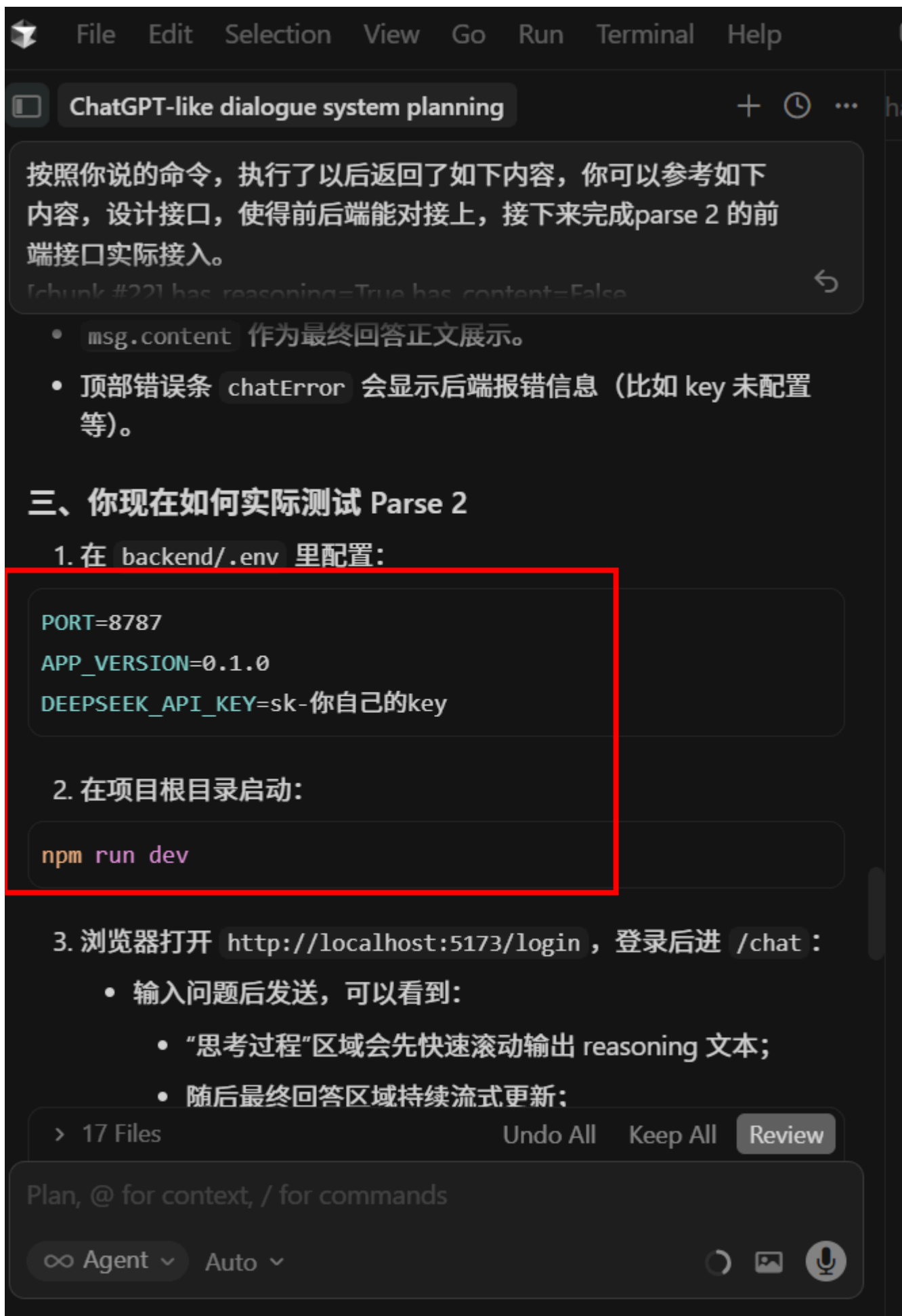
359 所以最终答案：3。

360

361 [本轮最终回答 content]

362 3。单词 strawberry 拼写为 s-t-r-a-w-b-e-r-r-y，其中字母 r 出现了三次。

运行完成以后，根据提示进行测试。



安装步骤测试后，发现基本能用，但是有一些不满足，继续交互优化即可。

目前可以完成基本的对话了。



4.完成 parse 3 任务开发

到目前为止项目基本可以用了，我们继续让 Cursor 往下执行即可。

参考提示词：

代码块

1 继续完成 parse 3 的任务开发，完成前后端接口联调。完成以后让我登录页面测试。

开发完后项目基本可用了。



5.完成功能优化

第一、功能优化

参考提示词：

代码块

1

功能基本可用，但是前端选择推理模型的时候，没有展示推理过程，而且遇到markdown的格式的时候，没有友好的展示

2

比如应该加粗的字符，显示的确实markdown的**，请重新优化一下。

优化后



第二、前端优化

参考提示词：

代码块

1

功能没问题了，我觉得这个UI太丑了，帮我优化一下，添加动态粒子背景，要有科技感，显的高大上一点

我觉得这个UI太丑了，帮我优化一下，添加动态粒子背景，要有科技感，显的高大上一点

Planning next moves

> 17 Files

Stop Ctrl+Shift+⌘

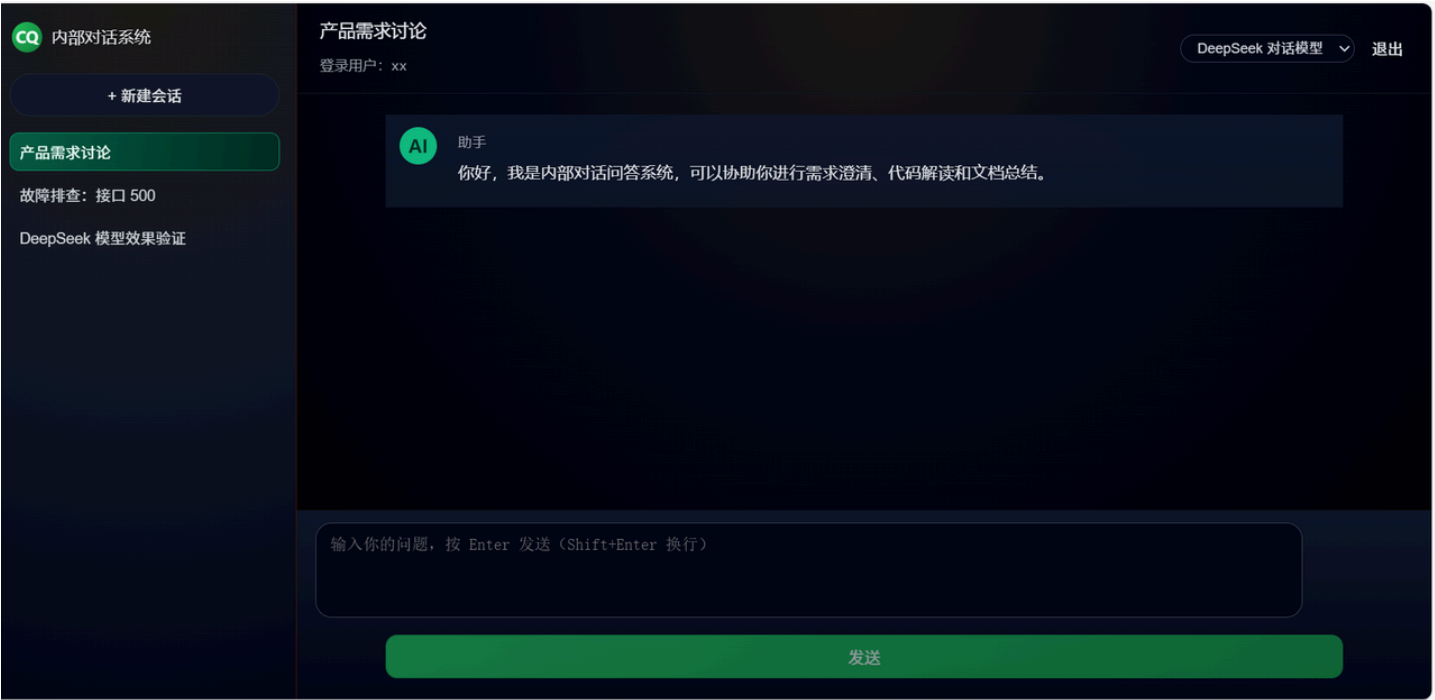
Review

Add a follow-up

∞ Agent ▾ Auto ▾



执行完以后，看一眼最后的效果！



大家可继续交互优化体验即可！

5、总结

本文详细讲解了借助 Cursor 工具从零开发类 ChatGPT 智能问答平台的全流程，核心遵循**梳理需求 - 拆分需求 - 分步开发 - 功能优化**的逻辑推进。首先在 Cursor 的 Plan 模式下明确公司内部使用的用户场景，规划技术架构并将需求拆分为四个核心阶段，确定各阶段开发重点。随后切换至 Agent 模式逐步开发，先完成项目初始化与接口健康度测试，再开发前端 UI 并模仿 ChatGPT 风格优化布局，接着通过 Python 脚本测试 DeepSeek 模型接口数据格式，实现后端接口与模型的对接及前后端打通，最后完成推理和对话双模型集成。开发中针对推理过程不展示、Markdown 格式渲染异常等问题优化功能，还可根据需求为界面添加动态粒子背景等特效，通过持续交互调试完善平台体验。